학습 가이드 (Learning Guide)

OS 스케줄링 개념과 LLM 스케줄링 적용

작성일: 2026-01-25 버전: 1.0.0

목차

1. 서론
2. OS 스케줄링 기초
3. LLM 스케줄링 적용
4. 알고리즘별 학습
5. 실습 예제
6. 연습 문제

---

1. 서론

1.1 학습 목표

이 가이드를 통해 다음을 학습합니다:

- 운영체제의 프로세스 스케줄링 기본 개념 이해
- 스케줄링 알고리즘의 동작 원리 파악
- LLM API 요청 관리에 스케줄링 적용 방법 학습
- 실제 코드를 통한 구현 경험

1.2 선수 지식

- 프로그래밍 기초 (TypeScript/JavaScript 권장)
- 자료구조 기초 (큐, 힙)
- 기본적인 시스템 프로그래밍 개념

---

2. OS 스케줄링 기초

2.1 스케줄링이란?

스케줄링**(Scheduling)**은 제한된 자원(CPU)을 여러 프로세스가 공유할 때, 어떤 프로세스에게 자원을 할당할지 결정하는 것입니다.

```
[프로세스 A]
[프로세스 B]  --> [스케줄러] --> [CPU] --> 결과
[프로세스 C]
```

2.2 스케줄링 기준

1. **CPU 이용률 (CPU Utilization)**: CPU가 얼마나 바쁘게 동작하는가
2. **처리량 (Throughput)**: 단위 시간당 완료되는 프로세스 수
3. **총처리 시간 (Turnaround Time)**: 요청 제출부터 완료까지 걸린 시간
4. **대기 시간 (Waiting Time)**: Ready Queue에서 대기한 시간
5. **응답 시간 (Response Time)**: 요청 제출부터 첫 응답까지 시간

2.3 프로세스 상태

```
          +--------+
          |  생성  |
          +---+----+
              |
```

```
                v
+--------+   +--------+   +--------+
| 대기중 |-->| 실행중 |-->|  종료  |
| (Ready)|   |(Running)|   |(Exit) |
+----+---+   +----+---+   +--------+
     ^            |
     |            v
     |       +---------+
     +-------| 블로킹  |
             |(Blocked)|
             +---------+
```

2.4 선점형 **vs** 비선점형

비선점형 **(Non-preemptive)** - 프로세스가 CPU를 자발적으로 반납할 때까지 계속 실행 - 구현이 단순 - 예: FCFS

선점형 **(Preemptive)** - 스케줄러가 강제로 CPU를 회수할 수 있음 - 응답성이 좋음 - 예: Round Robin, MLFQ (부분적)

---

3. LLM 스케줄링 적용

3.1 개념 매핑

OS 개념	LLM 스케줄링
프로세스	LLM API 요청
CPU	LLM 서비스 (API 호출 권한)
Ready Queue	BullMQ 대기열
프로세스 상태	요청 상태 (Pending, Queued, Processing, Completed)
Time Quantum	요청 처리 시간 제한
Context Switch	요청 전환 (한 요청 완료 후 다음 요청 처리)

3.2 왜 **LLM**에 스케줄링이 필요한가?

1. 비용 관리: API 호출 비용이 높음
2. 자원 제한: 동시 요청 수 제한 (Rate Limiting)
3. 응답성: 급한 요청은 빨리 처리해야 함
4. 공정성: 모든 사용자에게 공평한 서비스

3.3 실제 적용 시나리오

시나리오 **1:** 고객 서비스 챗봇

```
VIP 고객 요청     --> 높은 우선순위
일반 고객 요청  --> 보통 우선순위
내부 테스트      --> 낮은 우선순위
```

시나리오 **2:** 멀티테넌트 **SaaS**

```
Enterprise 고객 --> 가중치 100 (대역폭의 ~58%)
Premium 고객   --> 가중치 50 (대역폭의 ~29%)
Free 고객      --> 가중치 1 (대역폭의 ~0.6%)
```

---

4. 알고리즘별 학습

4.1 FCFS (First-Come, First-Served)

4.1.1 원리

가장 먼저 도착한 요청을 가장 먼저 처리합니다.

```
도착 순서: A(0ms) -> B(1ms) -> C(2ms)
처리 시간: A=5ms, B=3ms, C=2ms

타임라인:
|--A(5ms)--|--B(3ms)--|--C(2ms)--|
0          5          8          10
```

4.1.2 성능 계산

```
대기 시간:
- A: 0ms (즉시 시작)
- B: 5ms (A가 끝날 때까지)
- C: 8ms (A, B가 끝날 때까지)
평균 대기 시간: (0 + 5 + 8) / 3 = 4.33ms

총처리 시간:
- A: 5ms
- B: 8ms (도착 후 7ms 후 완료)
- C: 10ms (도착 후 8ms 후 완료)
평균 총처리 시간: (5 + 7 + 8) / 3 = 6.67ms
```

4.1.3 핵심 코드

```
class FCFSScheduler {
  private queue: Queue;

  async submit(request: LLMRequest): Promise<string> {
    // 큐의 끝에 추가 (도착 순서 유지)
    const job = await this.queue.add(request.id, request);
    return job.id;
  }
}
```

4.2 Priority Scheduling

4.2.1 원리

각 요청에 우선순위를 부여하고, 높은 우선순위 요청을 먼저 처리합니다.

```
요청: A(LOW), B(HIGH), C(URGENT), D(NORMAL)

우선순위 큐:
C(URGENT) -> B(HIGH) -> D(NORMAL) -> A(LOW)

처리 순서: C -> B -> D -> A
```

4.2.2 기아 문제와 에이징

문제: 낮은 우선순위 요청이 영원히 처리되지 않을 수 있음

해결 (에이징):

```
시간 0: A(LOW, 0번 승격)
시간 2분: A 대기 > 임계값 -> A(NORMAL, 1번 승격)
```

```
시간 4분: A 대기 > 임계값 -> A(HIGH, 2번 승격)
시간 6분: A는 HIGH 유지 (최대 승격 횟수 도달)
```

4.2.3 핵심 코드

```
class PriorityScheduler {
  async submit(request: LLMRequest): Promise<string> {
    // BullMQ에서 낮은 숫자가 높은 우선순위
    const priority = (3 - request.priority) * 2;

    const job = await this.queue.add(request.id, request, {
      priority: priority
    });
    return job.id;
  }
}
```

4.3 MLFQ (Multi-Level Feedback Queue)

4.3.1 원리

1. 여러 개의 우선순위 큐 사용
2. 새 요청은 최고 우선순위 큐에서 시작
3. 시간을 많이 사용하면 낮은 큐로 강등
4. 주기적으로 모든 요청을 최고 큐로 부스팅

```
+--------+  타임 퀀텀: 1초
|   Q0   | <-- 새 요청 입장
+---+----+
    |
    | 시간 초과
    v
+--------+  타임 퀀텀: 3초
|   Q1   |
+---+----+
    |
    | 시간 초과
    v
+--------+  타임 퀀텀: 8초
|   Q2   |
+---+----+
    |
    | 시간 초과
    v
+--------+  타임 퀀텀: 무제한
|   Q3   |
+--------+

[5초마다 부스팅] --> 모든 요청을 Q0로 이동
```

4.3.2 동작 예시

```
요청 A: 처리 시간 500ms (짧은 요청)
요청 B: 처리 시간 5000ms (긴 요청)

A의 경로: Q0(500ms) -> 완료 (Q0에서 처리 완료)
B의 경로: Q0(1000ms) -> Q1(3000ms) -> Q2(1000ms) -> 완료

결과: 짧은 요청 A가 빠르게 처리됨
```

4.3.3 핵심 코드

```
class MLFQScheduler {
  private queues: Queue[] = []; // Q0, Q1, Q2, Q3

  async submit(request: LLMRequest): Promise<string> {
    // Rule 3: 새 요청은 Q0에서 시작
    const job = await this.queues[0].add(request.id, {
      ...request,
      queueLevel: 0,
      timeSliceRemaining: 1000, // Q0 타임 퀀텀
    });
    return job.id;
  }

  async demoteJob(requestId: string, currentLevel: number): Promise<void> {
    // Rule 4: 시간 초과 시 강등
    const newLevel = Math.min(currentLevel + 1, 3);
    // 현재 큐에서 제거 후 다음 큐로 이동
    await this.queues[newLevel].add(requestId, {
      queueLevel: newLevel,
      timeSliceRemaining: TIME_QUANTA[newLevel],
    });
  }
}
```

4.4 WFQ (Weighted Fair Queuing)

4.4.1 원리

1. 각 테넌트에게 가중치 부여
2. 가상 시간(Virtual Time)으로 공정성 추적
3. 가상 종료 시간이 가장 작은 요청 먼저 처리

```
테넌트 A: 가중치 100 (Enterprise)
테넌트 B: 가중치 10 (Standard)

대역폭 분배: A는 B보다 10배 많은 서비스 받음
```

4.4.2 가상 시간 계산

```
가상 종료 시간 공식:
F_i = max(V(t), F_i_prev) + (ServiceTime / Weight)

예시:
- 현재 가상 시간 V(t) = 100
- 테넌트 A의 이전 종료 시간 = 90
- 요청의 서비스 시간 = 1000ms
- 테넌트 A의 가중치 = 100

F_A = max(100, 90) + (1000 / 100) = 100 + 10 = 110
```

4.4.3 공정성 측정 (Jain's Fairness Index)

```
J = (sum x_i)^2 / (n * sum x_i^2)

예시:
3명의 테넌트가 각각 [100, 100, 100]의 서비스를 받음
sum = 300
sum of squares = 30000
```

```
J = (300)^2 / (3 * 30000) = 90000 / 90000 = 1.0 (완벽한 공정성)

3명의 테넌트가 각각 [200, 100, 0]의 서비스를 받음
sum = 300
sum of squares = 50000
J = (300)^2 / (3 * 50000) = 90000 / 150000 = 0.6 (불공정)
```

---

5. 실습 예제

5.1 환경 설정

```bash
# 프로젝트 클론
git clone https://github.com/your-repo/llm-scheduler.git
cd llm-scheduler

# 의존성 설치
npm install

# 인프라 서비스 시작
docker-compose up -d

# 개발 서버 시작
npm run dev
```

5.2 예제 1: FCFS 테스트

```bash
# 순차적으로 3개 요청 전송
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Request 1", "priority": 1}'

curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Request 2", "priority": 1}'

curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Request 3", "priority": 1}'

# 대기열 상태 확인
curl http://localhost:3000/api/queue/status

# 예상 결과: Request 1 -> 2 -> 3 순서로 처리
```

5.3 예제 2: Priority 테스트

```bash
# 스케줄러를 Priority로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "priority"}'

# 다양한 우선순위로 요청 전송
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Low priority", "priority": 0}'

curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
```

```
  -d '{"prompt": "Urgent!", "priority": 3}'

curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Normal", "priority": 1}'

# 예상 결과: Urgent -> Normal -> Low 순서로 처리
```

5.4 예제 3: MLFQ 테스트

```
# 스케줄러를 MLFQ로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "mlfq"}'

# 짧은 요청과 긴 요청 전송
# 짧은 요청: 간단한 프롬프트
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Hi", "priority": 1}'

# 긴 요청: 복잡한 프롬프트
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Write a detailed essay about quantum computing...", "priority": 1}'

# 예상 결과: 짧은 요청이 Q0에서 빠르게 완료, 긴 요청은 강등됨
```

---

6. 연습 문제

6.1 기초 문제

문제 **1**: 다음 요청들의 FCFS 평균 대기 시간을 계산하세요.

요청	도착 시간	처리 시간
A	0ms	10ms
B	2ms	5ms
C	4ms	8ms

정답 보기 처리 순서: A -> B -> C

대기 시간: - A: 0ms (도착 즉시 시작) - B: 10ms - 2ms = 8ms (A 완료까지 대기) - C: 15ms - 4ms = 11ms (A, B 완료까지 대기)

평균 대기 시간: (0 + 8 + 11) / 3 = 6.33ms 문제 **2**: 동일한 요청들에 대해 SJF(Shortest Job First)를 적용하면 평균 대기 시간은?

정답 보기 처리 순서: A -> B -> C (A가 먼저 도착해서 처리, B(5ms)가 C(8ms)보다 짧음)

대기 시간: - A: 0ms - B: 10ms - 2ms = 8ms - C: 15ms - 4ms = 11ms

평균 대기 시간: (0 + 8 + 11) / 3 = 6.33ms

(이 경우 FCFS와 같음. 왜냐하면 A가 먼저 도착해서 이미 처리 중이기 때문)

6.2 중급 문제

문제 **3**: MLFQ에서 다음 시나리오의 최종 큐 레벨을 예측하세요.

- 요청 X: 처리 시간 500ms
- 요청 Y: 처리 시간 4000ms
- Q0 타임 퀀텀: 1000ms
- Q1 타임 퀀텀: 3000ms
- Q2 타임 퀀텀: 8000ms

정답 보기 요청 X: - Q0에서 시작, 500ms 처리 완료 - 최종 레벨: Q0

요청 Y: - Q0에서 시작, 1000ms 처리 후 타임 초과 -> Q1로 강등 - Q1에서 3000ms 처리 완료 (누적 4000ms) - 최종 레벨: Q1 문제 **4**: WFQ에서 가상 종료 시간을 계산하세요.

- 현재 가상 시간: V(t) = 50
- 테넌트 A의 이전 종료 시간: 45
- 테넌트 B의 이전 종료 시간: 60
- 새 요청 서비스 시간: 100ms
- 테넌트 A 가중치: 50
- 테넌트 B 가중치: 10

정답 보기 테넌트 A의 가상 종료 시간: F_A = max(50, 45) + (100 / 50) = 50 + 2 = 52

테넌트 B의 가상 종료 시간: F_B = max(50, 60) + (100 / 10) = 60 + 10 = 70

결론: 테넌트 A의 요청이 먼저 처리됨 (F_A=52 < F_B=70)

6.3 심화 문제
문제 **5**: 다음 시나리오에서 각 알고리즘별 성능을 비교하세요.

요청 패턴: - 50%는 짧은 요청 (100ms) - 30%는 중간 요청 (2000ms) - 20%는 긴 요청 (10000ms)

각 알고리즘 (FCFS, Priority, MLFQ)의 장단점을 분석하세요.

정답 보기 **FCFS:** - 장점: 구현 단순, 공정함 - 단점: 긴 요청이 짧은 요청을 블로킹 (Convoy Effect) - 평균 대기 시간: 높음 (긴 요청 때문에)

Priority: - 장점: 중요한 요청 우선 처리 가능 - 단점: 짧은/긴 구분 없이 우선순위만 봄 - 평균 대기 시간: 우선순위 분포에 따라 다름

MLFQ: - 장점: 짧은 요청 빠르게 처리 (Q0에서 완료) - 장점: 긴 요청도 결국 처리 (Q3에서) - 장점: 자동 적응 (워크로드 예측 불필요) - 단점: 구현 복잡도 높음 - 평균 대기 시간: 가장 낮음 (짧은 요청 최적화)

결론: 이 워크로드에는 MLFQ가 가장 적합

추가 학습 자료

참고 서적
- “Operating Systems: Three Easy Pieces” (OSTEP) - MLFQ 챕터
- “Operating System Concepts” (공룡책) - 스케줄링 챕터

온라인 자료
- OSTEP 무료 온라인 버전: https://pages.cs.wisc.edu/~remzi/OSTEP/
- BullMQ 공식 문서: https://docs.bullmq.io/

문서 버전: 1.0.0 최종 업데이트: 2026-01-25 작성자: LLM Scheduler 팀